

Week 4. Primary CNI

Primary CNI는 Pod에 네트워크 인터페이스와 IP를 붙이고, Pod 간 통신 경로를 만드는 기본 CNI이다.

1. Overlay vs Underlay

구분	Overlay	Underlay (Native Routing)
방식	Pod 패킷을 Node IP 패킷으로 한 번 더 감싸서(캡슐화) 전송한다	Pod IP를 그대로 물리 네트워크에서 라우팅한다
대표 기술	VXLAN, IP-in-IP	host-gw, BGP 기반 라우팅
물리 네트워크 요구 사항	Node 간 IP 통신만 되면 된다	물리 네트워크(또는 Node)가 Pod CIDR 경로를 알아야 한다
오버헤드	헤더 추가로 MTU 감소, 캡슐화 비용이 있다	캡슐화가 없어 오버헤드가 적다
트러블슈팅	패킷이 감싸져 있어 추적이 상대적으로 어렵다	일반 라우팅과 같아 추적이 상대적으로 쉽다

- Overlay는 기존 네트워크를 크게 변경하지 않고 구성할 수 있어 도입이 쉽다.
- Underlay는 캡슐화가 없어 오버헤드가 적지만, Pod CIDR에 대한 라우팅 구성이 필요하다.

2. Flannel

2.1 구성 요소

구성 요소	역할
<code>flanneld</code>	각 Node에서 실행되며 Node별 Pod 서브넷을 관리하고 Backend를 설정한다. Kubernetes에서는 일반적으로 DaemonSet으로 실행된다
<code>flannel</code> CNI 플러그인	Pod 생성 시 호출되어 네트워크 인터페이스와 IPAM 등의 구성을 수행한다
<code>cni0</code>	일반적인 구성에서 Pod들이 연결되는 Linux bridge이다. 실제 이름은 설정에 따라 달라질 수 있다
<code>flannel.1</code>	VXLAN Backend 사용 시 생성되는 VXLAN 인터페이스이다

- 클러스터 CIDR(예: `10.244.0.0/16`)을 Node별 Pod 서브넷으로 나눠 할당한다. 일반적인 예에서는 `/24`를 사용할 수 있지만 실제 크기는 설정에 따라 달라진다.

- 서브넷 정보는 Kubernetes API의 Node `podCIDR` 등을 통해 관리할 수 있다.

2.2 Backend

Backend	방식	특징
<code>vxlan</code>	Overlay	VXLAN을 사용하여 Pod 트래픽을 캡슐화한다
<code>host-gw</code>	Underlay	상대 Node IP를 게이트웨이로 하는 정적 라우트를 추가한다. Node들이 같은 L2 네트워크에 있어야 한다
<code>wireguard</code>	Overlay	암호화된 터널을 사용한다
<code>udp</code>	Overlay	UDP를 이용한 터널 방식으로, 일반적으로 운영 환경에서는 VXLAN이나 host-gw 등이 우선 고려된다

VXLAN Backend에서는 일반적으로 UDP `8472` 포트를 사용한다.

2.3 VXLAN 동작 요약

1. Pod 패킷이 `cni0` → Node 라우팅 테이블을 거쳐 `flannel.1` 로 전달된다.
 2. `flannel.1` 이 원본 패킷을 VXLAN 헤더 + UDP + 바깥 IP(목적지 Node IP)로 감싼다.
 3. 목적지 Node의 `flannel.1` 이 캡슐을 벗기고 `cni0` 를 거쳐 Pod로 전달한다.
- `flanneld` 가 Node별 Pod 서브넷과 Backend 구성을 관리하면 Linux 커널의 라우팅 테이블과 VXLAN FDB 등의 네트워크 상태가 구성된다.

3. Calico

3.1 구성 요소

구성 요소	역할
<code>calico-node</code>	각 Node에서 실행되는 Calico의 핵심 구성 요소이다. 일반적으로 DaemonSet으로 실행된다
Felix	라우트, ACL(iptables/nftables/eBPF) 등 Node의 데이터 플레인을 프로그래밍한다
BIRD	BGP를 사용하는 구성에서 BGP 경로를 교환한다
confd	데이터 변경을 감지해 관련 설정을 갱신한다
<code>calico-kube-controllers</code>	Kubernetes 리소스 변화를 Calico 데이터와 동기화한다
Typha (선택)	대규모 클러스터에서 데이터 저장소 부하를 줄이는 캐시 역할을 한다

구성 요소	역할
<code>calico</code> / <code>calico-ipam</code> CNI 플러그인	Pod 인터페이스 생성과 IP 할당을 담당한다

3.2 네트워크 구성 특징

- 일반적인 Calico routed mode에서는 bridge를 사용하지 않는다. Pod의 veth(`cali*`)가 Node 네트워크 네임스페이스에 연결되고 Node의 라우팅 테이블을 통해 패킷을 전달한다.
- Node에 Pod IP별 `/32` 라우트가 추가될 수 있다.
- Pod 안의 기본 게이트웨이는 `169.254.1.1`이며, Node 쪽에서 Proxy ARP로 응답하도록 구성할 수 있다.
- 결과적으로 일반적인 routed mode에서는 같은 Node 안의 통신도 L2 스위칭이 아니라 L3 라우팅으로 처리된다.

3.3 Routing / BGP

- 각 Node는 자신에게 할당된 IP 블록 경로를 BGP로 다른 Node(또는 라우터)에 광고할 수 있다.
- 전통적인 Calico BGP 구성에서는 Node 간 Full Mesh를 구성할 수 있다. Node 수가 많아지면 세션 수가 증가하므로 Route Reflector를 사용할 수 있다.
- 물리 라우터(ToR)와 BGP Peering을 맺어 외부 네트워크에서 Pod 네트워크 경로를 학습하게 만들 수도 있다.

3.4 Encapsulation 옵션

모드	방식	인터페이스
없음 (Native)	BGP 등으로 받은 경로를 이용해 바로 라우팅한다	-
IP-in-IP	IP 패킷을 IP 헤더로 한 번 더 감싼다	<code>tunl0</code>
VXLAN	UDP 기반으로 캡슐화한다	<code>vxlan.calico</code>

- IPIP/VXLAN 모드는 `Always`, `CrossSubnet`, `Never` 등의 방식으로 설정할 수 있다.
- `CrossSubnet`은 같은 서브넷 Node끼리는 Native 라우팅, 다른 서브넷으로 갈 때만 캡슐화를 사용하는 방식이다.

4. Flannel vs Calico 비교

항목	Flannel	Calico
대표적인 방식	Overlay (VXLAN)	Native routing, 필요 시 IPIP/VXLAN
Node 내부 연결	<code>cni0</code> bridge (L2)	일반적인 routed mode에서는 bridge 없이 <code>/32</code> 라우트 + Proxy ARP (L3)
Node 간 경로 전파	<code>flannel</code> 이 네트워크 정보를 보고 설정	BGP 등을 이용해 경로를 광고할 수 있음
IPAM	일반적인 구성에서 <code>host-local</code> 사용	<code>calico-ipam</code> (IP Pool에서 블록 단위로 할당)
NetworkPolicy	자체적으로 지원하지 않음	지원함 (Kubernetes NetworkPolicy + Calico 확장 정책)
구성 난이도	단순하다	기능이 많은 만큼 설정 요소가 많다

5. Pod-to-Pod Packet Flow 비교

5.1 같은 Node

Flannel

```
Pod A (eth0) → veth → cni0 (bridge) → veth → Pod B (eth0)
```

- 같은 네트워크에 있는 경우 bridge에서 L2로 전달된다.

Calico

```
Pod A (eth0) → cali-A (Node) → Node 라우팅 테이블 (/32) → cali-B → Pod B (eth0)
```

- Pod A가 게이트웨이(`169.254.1.1`)를 통해 패킷을 보내면 Node가 목적지 Pod IP의 경로를 확인하고 Pod B의 veth로 전달한다.

5.2 다른 Node

Flannel (VXLAN)

```
Pod A → cni0 → Node1 라우팅 → flannel.1 (캡슐화)
→ eth0 —[UDP 8472, Node1 IP → Node2 IP]→ eth0
→ flannel.1 (역캡슐화) → cni0 → Pod B
```

Calico (Native Routing)

```
Pod A → cali-A → Node1 라우팅 (경로 정보를 통해 Node2 블록 확인)
→ eth0 —[Pod A IP → Pod B IP 그대로]→ eth0
→ Node2 라우팅 → cali-B → Pod B
```

Calico (IPIP)

```
Pod A → cali-A → Node1 라우팅 → tunl0 (IP-in-IP 캡슐화)
→ eth0 —[Node1 IP → Node2 IP]→ eth0
→ tunl0 (역캡슐화) → cali-B → Pod B
```

- 같은 Pod A → Pod B 통신이어도 bridge를 거치는지, 캡슐화가 되는지, 물리 네트워크에 Pod IP가 그대로 보이는지가 CNI와 모드에 따라 달라진다.

6. NetworkPolicy / IPAM 처리 위치

6.1 IPAM

항목	내용
처리 시점	Pod 생성 시 CNI 플러그인이 호출되는 과정에서 처리된다
처리 주체	CNI 설정의 <code>ipam</code> 항목에 지정된 IPAM 플러그인
Flannel	일반적인 구성에서 <code>host-local</code> 이 Node에 할당된 Pod 서브넷 안에서 IP를 고르고, 할당 정보를 Node 로컬에 저장한다
Calico	<code>calico-ipam</code> 이 IP Pool에서 IP를 할당하고 Node별 IP 블록을 관리할 수 있다

6.2 NetworkPolicy

항목	내용
정의	Kubernetes API 리소스(<code>networking.k8s.io/v1</code>)로 선언한다
적용	실제 차단/허용은 CNI(또는 정책 엔진)가 구현해야 한다. 지원하지 않는 CNI에서는 리소스를 만들어도 효과가 없다
계층	주로 L3/L4 (IP, Port, Protocol) 기준으로 동작한다
Calico	Felix가 정책을 해석하고 iptables/nftables/eBPF 등의 데이터 플레인에 적용한다
Flannel	기본 Flannel 자체는 NetworkPolicy 구현을 제공하지 않는다. 필요하면 별도의 정책 기능을 함께 사용할 수 있다

- 정리하면 IPAM은 Pod 생성 시 CNI IPAM 플러그인 단계에서, NetworkPolicy는 Node의 데이터 플레인에서 처리된다.

7. 핵심 정리

- Flannel은 Node별 서브넷 + VXLAN Overlay를 이용해 Pod 통신을 단순하게 제공하는 CNI이다.
- Calico는 L3 라우팅을 중심으로 하며 IPAM과 NetworkPolicy까지 함께 제공한다.
- 다만 Calico도 IPIP/VXLAN 등의 캡슐화를 사용할 수 있기 때문에 Flannel=Overlay, Calico=Underlay로 단순하게 구분하면 안 된다.
- 같은 Node 통신은 일반적인 Flannel 구성에서 bridge를 통한 L2 forwarding으로, Calico routed mode에서는 L3 라우팅으로 처리된다.
- Overlay는 구성이 쉽지만 캡슐화 오버헤드가 있고, Underlay는 캡슐화가 없어 오버헤드가 적지만 네트워크 라우팅 구성이 필요하다.
- IPAM은 Pod 생성 시 CNI IPAM 플러그인이, NetworkPolicy는 CNI의 정책 엔진이 데이터 플레인에서 처리한다.

질문

1. Calico는 왜 bridge 대신 /32 라우트와 Proxy ARP 방식을 사용하는지?

- 일반적인 routed mode에서 모든 Pod 트래픽을 L3 라우팅 중심으로 처리하기 위해서이다.
- bridge를 사용하지 않고 Node 라우팅 테이블을 통해 Pod 간 통신을 처리할 수 있다.
- /32 라우트로 Pod마다 개별 경로를 가질 수 있다.
- Pod의 게이트웨이 169.254.1.1에 대해 Node 쪽 veth가 Proxy ARP로 응답하여 Node가 게이트웨이 역할을 한다.
- L2 브로드캐스트를 사용하는 bridge 방식 대신 L3 라우팅을 사용하지만, Pod 수가 증가하면 관리해야 할 라우팅 경로도 늘어난다.

2. NetworkPolicy와 kube-proxy의 iptables 규칙은 어떤 순서로 적용되는지?

- kube-proxy의 DNAT → NetworkPolicy 순서라고 모든 환경에 대해 단정할 수는 없다.
- kube-proxy는 Service IP를 실제 Endpoint로 전달하기 위한 규칙을 구성하고, NetworkPolicy는 CNI 또는 정책 엔진이 구현한다.

- 실제 정책 평가 시점과 어떤 주소/포트를 기준으로 평가하는지는 CNI와 데이터 플레인 구현에 따라 달라질 수 있다.
 - 따라서 항상 DNAT가 먼저 일어나고 NetworkPolicy가 그 다음에 적용된다거나 항상 targetPort를 기준으로 정책이 평가된다고 일반화할 순 없다.
-

▼ 참고 자료

- Kubernetes Network Plugins:
<https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>
- Flannel Backends: <https://github.com/flannel-io/flannel/blob/master/Documentation/backends.md>
- Kubernetes Network Policies:
<https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- Kubernetes Virtual IPs:
<https://kubernetes.io/docs/reference/networking/virtual-ips/>
- Calico Networking: <https://docs.tigera.io/calico/latest/networking/>
- Calico Component Architecture:
<https://docs.tigera.io/calico/latest/reference/architecture/overview>